

DAA Lab

Name: Devansh Joshi

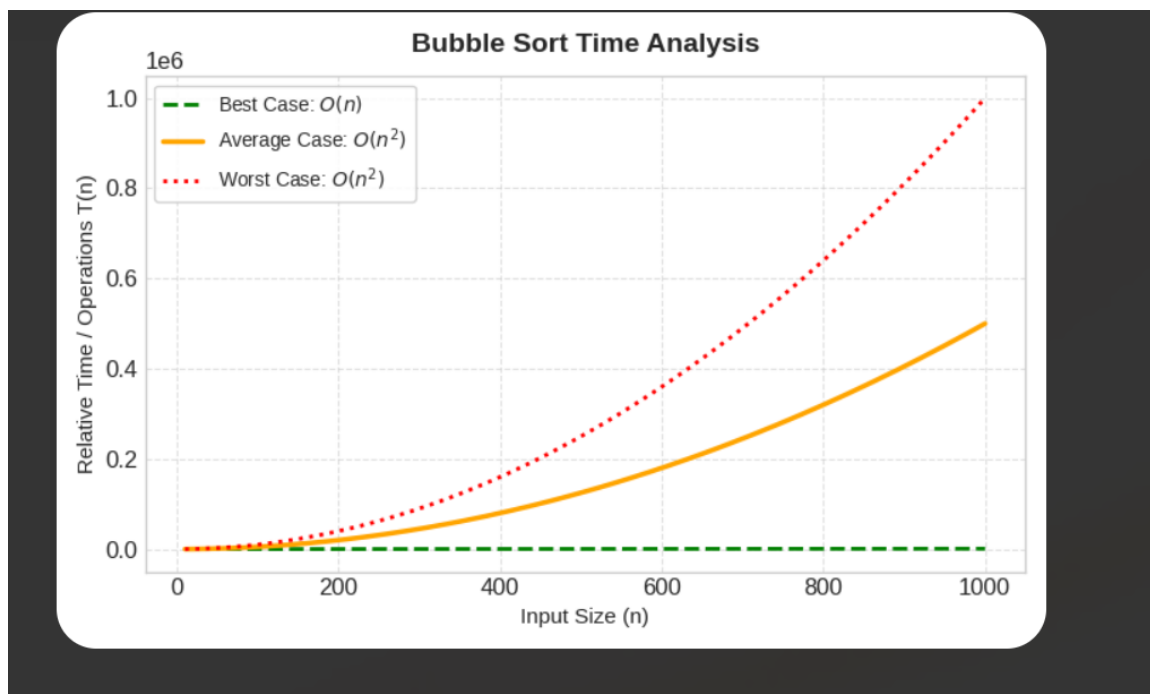
SAP ID: 590028189

1. Bubble Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > bubbleSort(int [], int)
1  #include <stdio.h>
2
3  void bubbleSort(int arr[], int n) {
4      for (int i = 0; i < n - 1; i++) {
5          int swapped = 0;
6          for (int j = 0; j < n - i - 1; j++) {
7              if (arr[j] > arr[j + 1]) {
8                  int temp = arr[j];
9                  arr[j] = arr[j + 1];
10                 arr[j + 1] = temp;
11                 swapped = 1;
12             }
13         }
14         if (!swapped) break; // Optimized for best-case O(n)
15     }
16 }
```

Time Complexity Graph:

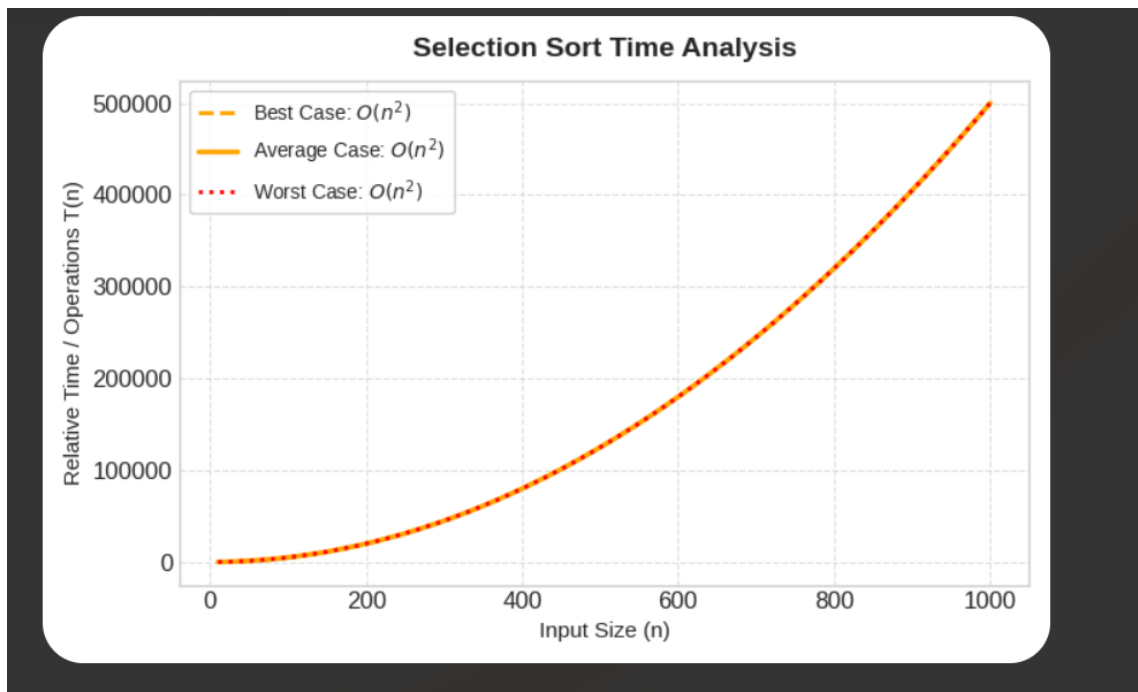


2. Selection Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > bubbleSort(int [], int)
1  #include <stdio.h>
2
3  void selectionSort(int arr[], int n) {
4      for (int i = 0; i < n - 1; i++) {
5          int minIdx = i;
6          for (int j = i + 1; j < n; j++) {
7              if (arr[j] < arr[minIdx]) minIdx = j;
8          }
9          int temp = arr[minIdx];
10         arr[minIdx] = arr[i];
11         arr[i] = temp;
12     }
13 }
```

Time Complexity Graph:

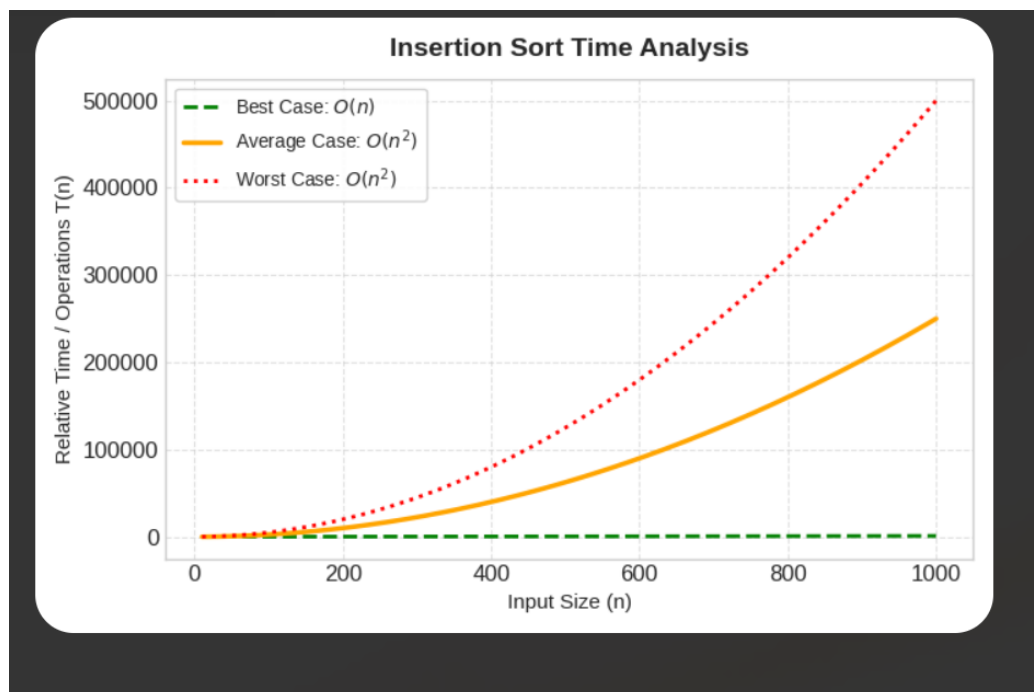


3. Insertion Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > selectionSort(int [], int)
1  #include <stdio.h>
2
3  void insertionSort(int arr[], int n) {
4      for (int i = 1; i < n; i++) {
5          int key = arr[i];
6          int j = i - 1;
7          while (j >= 0 && arr[j] > key) {
8              arr[j + 1] = arr[j];
9              j--;
10         }
11         arr[j + 1] = key;
12     }
13 }
```

Time Complexity Graph:

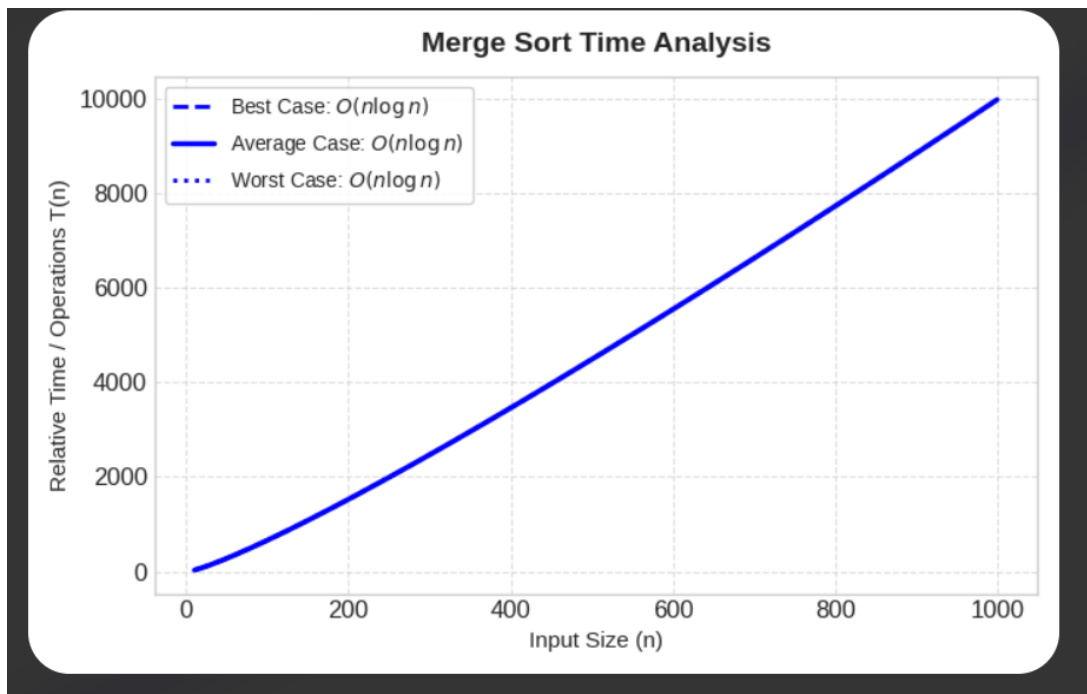


4. Merge Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > insertionSort(int [], int)
1  #include <stdio.h>
2
3  void merge(int arr[], int l, int m, int r) {
4      int n1 = m - l + 1, n2 = r - m;
5      int L[n1], R[n2];
6      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
7      for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
8
9      int i = 0, j = 0, k = l;
10     while (i < n1 && j < n2) {
11         if (L[i] <= R[j]) arr[k++] = L[i++];
12         else arr[k++] = R[j++];
13     }
14     while (i < n1) arr[k++] = L[i++];
15     while (j < n2) arr[k++] = R[j++];
16 }
17
18 void mergeSort(int arr[], int l, int r) {
19     if (l < r) {
20         int m = l + (r - l) / 2;
21         mergeSort(arr, l, m);
22         mergeSort(arr, m + 1, r);
23         merge(arr, l, m, r);
24     }
25 }
```

Time Complexity Graph:

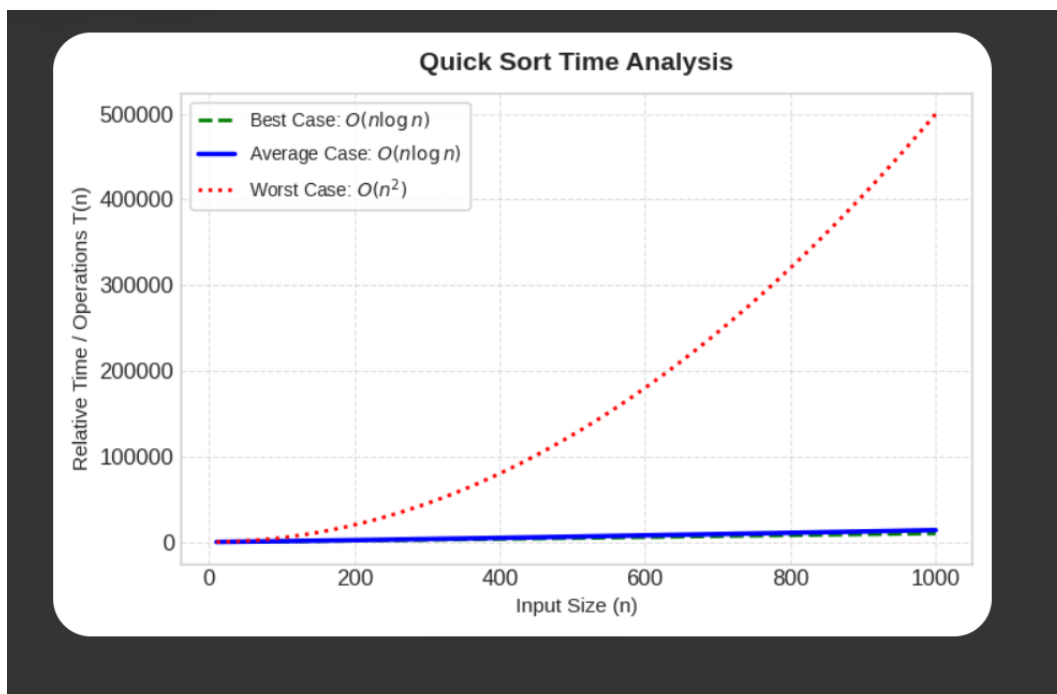


5. Quick Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > quickSort(int [], int, int)
1  #include <stdio.h>
2
3  int partition(int arr[], int low, int high) {
4      int pivot = arr[high], i = low - 1;
5      for (int j = low; j < high; j++) {
6          if (arr[j] < pivot) {
7              i++;
8              int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
9          }
10     }
11     int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
12     return i + 1;
13 }
14
15 void quickSort(int arr[], int low, int high) {
16     if (low < high) {
17         int pi = partition(arr, low, high);
18         quickSort(arr, low, pi - 1);
19         quickSort(arr, pi + 1, high);
20     }
21 }
```

Time Complexity Graph:

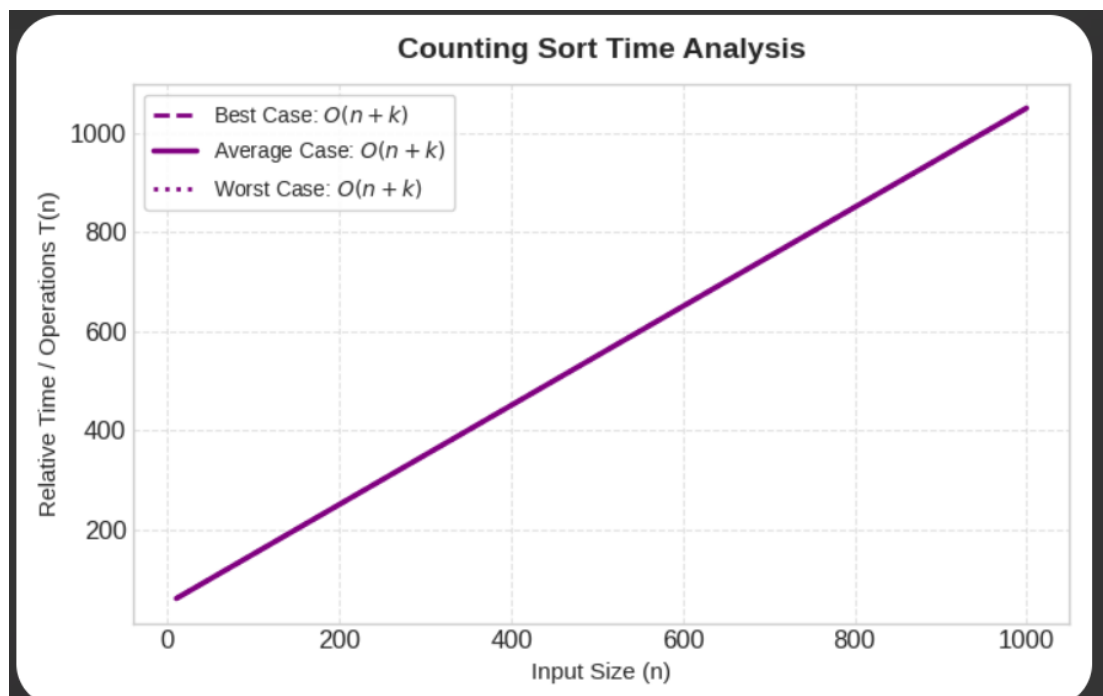


6. Counting Sort

Code:

```
C: > Users > devan > OneDrive > Desktop > DEMO.c > countingSort(int [], int)
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void countingSort(int arr[], int n) {
5      int max = arr[0];
6      for (int i = 1; i < n; i++) {
7          if (arr[i] > max) max = arr[i];
8      }
9
10     int *count = (int *)calloc(max + 1, sizeof(int));
11     for (int i = 0; i < n; i++) count[arr[i]]++;
12
13     int idx = 0;
14     for (int i = 0; i <= max; i++) {
15         while (count[i] > 0) {
16             arr[idx++] = i;
17             count[i]--;
18         }
19     }
20     free(count);
21 }
```

Time Complexity Graph:



7. Heap Sort

Time Complexity Graph:

